World-Model-Grounded LLM Planning for AUV and ASV Navigation Near Offshore Wind Farms

Markus Buchholz[1], Ignacio Carlucho[2] and Yvan R. Petillot[2]

***Abstract*—Large language models can turn a natural-language mission into a sequence of robot actions, but they do not have a sense of physics: they cannot judge how long a command should run, or whether it will make the robot drift into an obstacle. We proposed the use of a world model to expand the capabilities of Large Language model-based planners. Our method has three components: a physics-grounded neural world model, a three-phase gradient-based trajectory optimizer, and a Model Predictive Controller (MPC)-style closed-loop replanner with a trust-region guard. The language model decides *what* to do, and the world model decides *how long*, whether that means driving eight thrusters through 6 DOF or two differential thrusters through 3. We evaluate two marine vehicle classes operating near offshore wind infrastructure: a 6-DOF Autonomous Underwater Vehicle (AUV) and a 3-DOF differential-drive Autonomous Surface Vehicle (ASV). In five benchmark missions per platform, both vehicles reach every goal with zero predicted collisions, and both transfer to GazeboSim under ocean current, waves, and thruster dynamics, remaining collision-free and cutting GazeboSim goal-distance error versus the ungrounded baseline by 70-82% (ASV) and roughly 93% (AUV), after a residual fine-tuning pass that separately reduces surrogate rollout Root Mean Square Error (RMSE) by 60% (AUV) and 69% (ASV). For the ASV we further demonstrate a Vision language model (VLM)-assisted semantic-mapping pipeline that extracts obstacles and environmental context from satellite imagery, nautical charts, and forecast Application Programming Interface (API) instead of onboard sensors, reaching 96% navigability accuracy as a drop-in replacement for hand-specified obstacle geometry.**

I. Introduction

Large language models (LLM) have opened a compelling way to plan robot motion: describe a mission in natural language and let the LLM decompose it into executable actions [1]–[3]. This works well in manipulation and navigation, where commonsense reasoning about objects and space produces useful plans. Marine robotics near offshore wind infrastructure exposes a fundamental limitation of this approach, for both ASVs and AUVs. A command that looks geometrically reasonable, such as "move forward, then turn to avoid the turbine," can fail catastrophically for reasons the LLM cannot anticipate: thruster lag slows the first seconds of any move, hydrodynamic coupling makes combined surge and sway (or surge and yaw) unpredictable, and current or wind accumulate lateral drift over a mission. As LeCun argues [4], predicting the next token is the wrong objective for systems that must reason about the physical world, a language model that has processed millions of descriptions of marine vehicles still has no *representation* of what drag or thruster saturation does to a trajectory. It only knows how people *write* about these effects. LeCun's proposed alternative, JEPA [4], learns compact representations of future world states rather than token sequences; Ha and Schmidhuber [5] showed that an agent should *imagine* the consequences of actions in a compact model before committing, and Dreamer [6] proved this works for continuous control by planning entirely within a learned world model before any real action is taken. Fig. 1 captures the consequence for both vehicle classes: the same LLM planner that drives blindly into a subsea structure or a turbine tower reaches the goal cleanly once a world model checks and corrects its plan before execution. The LLM decides *what* to do in both cases, regardless of whether the vehicle has 6 DOF and eight thrusters or 3 DOF and two differential thrusters, and the world model decides *how long*, i.e., whether the plan will survive contact with the physics.

Two features of the problem are genuinely vehicle-specific. First, maneuverability: the AUV can move directly sideways to correct a slightly-off plan, while the ASV, like most surface vessels, is nonholonomic - its LLM must instead lay out an explicit turn-forward-turn sequence, and a wrong sequence cannot be corrected mid-execution (Section IV-C, IV-G). Second, knowing where the obstacles are is not free: our AUV missions assume obstacle positions from a pre-dive structural survey, but equipping a low-cost ASV with real-time sonar or LiDAR for complex coastal bathymetry is prohibitively expensive. For the ASV, we therefore also develop a vision-language model (VLM) pipeline that extracts obstacle and environmental context from satellite imagery, nautical charts, and forecast APIs - sensor-cheap data already available for Norwegian and North Sea coastal waters - in place of onboard perception.

In summary, this work contributes: **(1)** a physics-grounded neural surrogate - an analytical Fossen simulator plus a learned residual MLP - instantiated for both a 6-DOF AUV and a 3-DOF differential-drive ASV with sub-centimeter (AUV) and sub-$0.4\,\mathrm{m}$ (ASV) open-loop accuracy over $60\,\mathrm{s}$; **(2)** a shared three-phase gradient-based trajectory optimizer and MPC-style replanner with a trust-region guard, unchanged in structure across both vehicles, for which an ablation on the AUV's tightest slalom mission shows the guard is decisive (100% vs. 10% goal-reach without it); **(3)** a characterization of the skeleton-topology failure mode common to both platforms, including the ASV's differential-drive no-strafe constraint enforced at the LLM prompt level; **(4)** a VLM-assisted semantic-mapping pipeline for the ASV that extracts obstacles and environmental context from satellite imagery, nautical charts, and forecast

[1]Norwegian Defence Research Establishment (FFI), Kjeller, Norway markus.buchholz@ffi.no
[2]School of Engineering & Physical Sciences, Heriot-Watt University, Edinburgh, UK {ignacio.carlucho, y.r.petillot}@hw.ac.uk

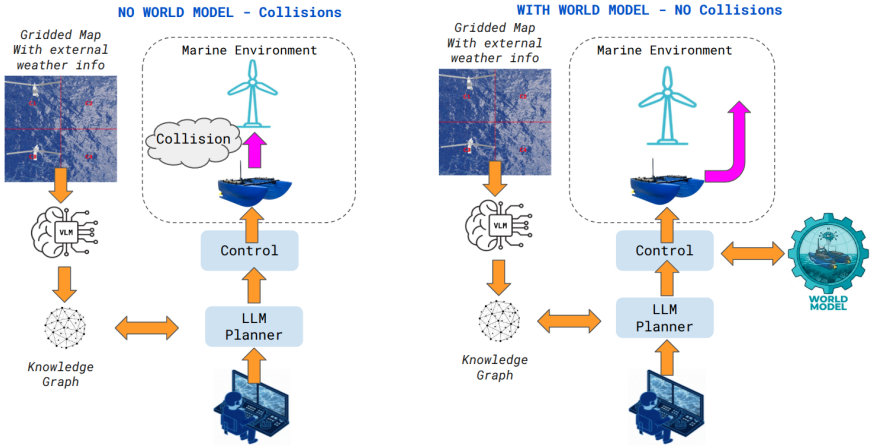


Fig. 1: Why an LLM planner needs a world model (ASV variant shown; the same contrast holds for the AUV, without the VLM stage). Without grounding (left), the LLM's plan drifts into the turbine tower; with the world model (right), the same plan receives physically feasible step durations and reaches the goal collision-free. The ASV's inputs additionally include a gridded map classified by a vision-language model and enriched with forecast wind/current/wave data (Section IV-F); the AUV instead uses pre-dive obstacle boxes (Section III), with no VLM stage.

data, validated as a drop-in replacement for hand-specified obstacle geometry; and **(5)** empirical evaluation on five missions per platform (3-D for the AUV, 2-D for the ASV) around offshore wind infrastructure, with transfer to GazeboSim under ocean currents, waves, and thruster dynamics.

The LLM is called once per mission; every subsequent duration-optimization and replanning step runs without querying it again.

II. Related Work

World models and LLM planning. Ha and Schmidhuber [5] showed that an agent should imagine consequences in a compact model before acting; Dreamer [6] proved this works for continuous control by planning entirely within a learned world model. LeCun [4] formalized the critique of token prediction for physical reasoning, arguing for architectures that predict *representations* of future states - our surrogate does exactly this for both vehicles. On the language side, SayCan [2] grounds LLM outputs in affordances, Code as Policies [7] generates executable programs, and Inner Monologue [8] and ELLMER [9] add feedback to the LLM loop; none insert a differentiable physics model between the LLM and the robot.

Physics-informed learning for marine vehicles. Neural surrogates exist as black-box models [10], [11] and for thrust allocation [12], discarding the analytical model entirely; hybrid first-principles-plus-residual models [13], [14] retain physical structure, which our surrogate follows for both vehicles. High-fidelity simulators including Stonefish [15], [16] and GazeboSim (the latter driven through ArduPilot, SITL and a ROS 2 bridge to replicate the real autopilot stack [17]) serve as digital twins; our surrogate plays a complementary role as a fast differentiable approximation for offline plan search. Broader agentic architectures for marine autonomy have also been explored [3], [18].

Semantic mapping for sensor-cheap navigation (ASV). Grid-based semantic mapping from satellite imagery has been demonstrated by OVerSeeC [19], and open-vocabulary scene graphs such as ConceptGraphs [20] support spatial retrieval for planning; our knowledge graph follows this design but merges cells directly into the differentiable collision field. How well LLMs cope with geospatial context is an open question: the Mil-SCORE benchmark shows current models struggle to combine maps, orders, and reports at scenario scale [21], which is why we hand the LLM a short, pre-digested terrain summary rather than raw map data.

III. Background: Unified Marine Vehicle Dynamics

We use the standard Fossen [22] notation throughout. In general, a marine vehicle's kinematics follow $\dot{\boldsymbol{\eta}} = \mathbf{R}(\boldsymbol{\eta})\boldsymbol{\nu}$, relating body-frame velocity $\boldsymbol{\nu}$ to the rate of change of world-frame pose $\boldsymbol{\eta}$ through the rotation (and, in 6-DOF, transformation) matrix $\mathbf{R}$. The kinetics follow

$$\mathbf{M}\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu}_r)\boldsymbol{\nu}_r + \mathbf{g}(\boldsymbol{\eta}) = \boldsymbol{\tau}, \tag{1}$$

where $\mathbf{M}$ is the generalized mass (rigid-body plus added mass), $\mathbf{C}$ is the Coriolis-centripetal matrix, $\mathbf{D}$ is hydrodynamic damping, $\mathbf{g}$ is the restoring vector, $\boldsymbol{\nu}_r = \boldsymbol{\nu} - \boldsymbol{\nu}_c$ is velocity relative to ambient current (or current-plus-wind), and $\boldsymbol{\tau}$ is the control wrench delivered through a first-order actuator lag with time constant τ_a. This single formulation specializes to each platform only in degrees of freedom, actuation, and coefficients (BlueROV2 Heavy [23], [24]; Blueboat [25]), summarized in Table I.

What makes the Blueboat's control wrench distinctive is that it is generated by only two collinear thrusters,

$$\boldsymbol{\tau} = \begin{bmatrix} T_p + c\,T_s \\ 0 \\ 0.5\,B\,(T_p - c\,T_s) \end{bmatrix}, \tag{2}$$

TABLE I: Platform specialization of Eq. (1).

	AUV (BlueROV2 Heavy)	ASV (Blueboat)
DOF	6, $\boldsymbol{\eta} = [x, y, z, \phi, \theta, \psi]$	3, $\boldsymbol{\eta} = [x, y, \psi]$
Actuation	8×T200, full $\mathbf{B}$	2×diff. thrusters, no-strafe
Mass	13.5 kg	16.0 kg
Planner state	$(\boldsymbol{\eta}, \boldsymbol{\nu}) \in \mathbb{R}^{12}$, SoC separate	$\mathbf{s} \in \mathbb{R}^{9}$, incl. (n_p, n_s), SoC
Actuator lag	first-order (validated, Sec. V-C)	first-order, τ_a=0.15 s, c=0.78

TABLE II: Surrogate input/output dimensionality per platform.

	AUV	ASV
Input dim.	50 (state, action, sin/cos angles, dt)	22 (norm. state, sin/cos yaw, norm. action/delta)
Output dim.	19-D state delta	9-D state delta (trainable zero-point)

where T_p, T_s are port/starboard thrust, B=0.41 m is the beam, and c=0.78 the measured starboard efficiency correction. This is the direct analogue of the AUV's full allocation-matrix wrench $\boldsymbol{\tau} = \mathbf{B}\mathbf{f}$, but with no lateral force component available at all, which is the physical source of the no-strafe constraint discussed in Section IV-C.

IV. Method

A. Surrogate World Model Architecture

The surrogate predicts the next state as

$$s_{t+1} = s_t + \Delta_{\text{physics}}(s_t, a_t) + \Delta_{\text{residual}}(s_t, a_t), \tag{3}$$

where Δ_{physics} is an analytical RK4 step from the Fossen simulator of Section III and Δ_{residual} is a neural-network correction - the physically-grounded world model Ha & Schmidhuber [5] and LeCun [4] argue for, instantiated in each vehicle's own state space. Three advantages follow from the hybrid design, for both platforms: the analytical half enforces physical limits that must always hold - angles wrap correctly, actuators saturate at their real limits, and battery charge only decreases; the network only corrects what the analytical model misses, requiring far less training data than an end-to-end model; and a large residual at an operating point signals where the analytical model is inaccurate. Obstacle geometry is never encoded in the surrogate; it enters only through the cost function (Section IV-D), so the same world model serves any mission without retraining.

The residual network is a four-layer MLP with hidden layers [512, 512, 256, 128], GELU activations, layer normalization, and dropout 0.05 for *both* vehicles; the two instantiations differ only in input/output dimensionality (Table II).

B. Training

Both surrogates use the same composite loss: a one-step prediction term, an absolute next-state term, an angle-aware term using sine/cosine features, a 50-step rollout term that forces long-horizon accuracy, an equilibrium term that removes trim-point drift, and a residual-magnitude regularizer that keeps the neural correction small and interpretable - optimized with Adam (lr 10^{-3}, batch 512, early stopping). Both training sets are generated from random smooth/step commands, stabilizing-controller rollouts, and macro-action sequences, with 25% of samples near equilibrium to ensure accurate passive-stabilization prediction. The AUV model trains on 500 k transitions and converges in 48 epochs (validation loss $\sim 10^{-4}$; final RMSE below 0.06 mm position, 0.3 mm/s velocity); the ASV model trains on 200 k transitions (loss weights w_1=1.0, w_2=0.5, w_3=2.0, w_4=0.5, w_{eq}=1.0, w_{res}=0.1; patience 15).

C. LLM Planning Interface

Both systems use `gemma4:26b`, served locally via Ollama at temperature 0.3, expose the LLM to a fixed macro-action vocabulary, and invoke it exactly once per mission: the LLM returns an ordered list of `(macro, duration)` pairs with no numeric durations attached, and the differentiable world model finds how long each step should run. The **AUV** interface exposes nine macros - `stabilize`, `move_forward/backward`, `sway_left/right`, `ascend`, `descend`, `turn_left/right` - covering full 3-D motion including direct lateral strafing. The **ASV** interface exposes five macros - `stabilize`, `move_forward/backward`, `turn_left/right` - with no sway macro at all: the Blueboat's two collinear thrusters (Eq. (2)) cannot generate a lateral force component, so the LLM must reason in turn-forward-turn sequences rather than direct lateral detours, and the no-strafe constraint is additionally stated explicitly in the prompt. Both interfaces score a candidate plan with the same cost, which the gradient optimizer of Section IV-D minimizes over the per-step durations proposed by the LLM:

$$J = d_{\text{goal}} + 0.1\, L_{\text{path}} + 100 \sum_t \phi_{\text{coll}}(\mathbf{p}_t), \tag{4}$$

where d_{goal} is final distance to goal, L_{path} is path length, and ϕ_{coll} is the smooth obstacle-membership field of Eq. (5).

D. Gradient-Based Trajectory Optimization

This step belongs to the gradient optimizer, not the LLM: taking the LLM's macro-action skeleton as fixed, it searches over the per-step durations to minimize Eq. (4). Because the residual network Δ_{residual} is differentiable, this search can follow gradients directly through the surrogate, while the frozen analytical component Δ_{physics} is treated as a constant at each step. Obstacles enter through a smooth sigmoid repulsion field

$$\phi_{\text{coll}}(\mathbf{p}) = \sum_{o \in \mathcal{O}} \prod_{k \in \{x,y[,z]\}} \sigma\Big(\alpha\big(h_k^{(o)} - |p_k - c_k^{(o)}|\big)\Big), \tag{5}$$

where $c_k^{(o)}, h_k^{(o)}$ are the center and half-extent of obstacle o along axis k and α=10 controls boundary sharpness; the AUV case takes the product over $k \in \{x, y, z\}$, while the ASV's planar workspace drops the z-term and takes the product over $\{x, y\}$ only. Optimization proceeds in three phases, identical in structure for both vehicles (Algorithm 1): *Phase 1* evaluates ten uniform duration scales $\{0.5\text{-}5.0\times\}$ and keeps the best;

Algorithm 1 Gradient-Optimized World-Model Planning

Require: Mission M, LLM C, world model W, candidates K

$\Pi \leftarrow$ parse(C.generate(build_prompt(M, K)))
$\Pi \leftarrow \Pi \cup$ geometric_candidates(M)
for each skeleton π in Π **do**
  Phase 1: $s^* \leftarrow \arg\min_{s \in \mathcal{S}} J(W.\text{sim}(M, \text{scale}(\pi, s)))$
  Phase 2: adaptive coordinate descent ($\Delta \in \{2.0 \ldots 0.05\}$)
  Phase 3: Adam gradient pass (differentiable W)
  $J_\pi \leftarrow$ best cost from phases 1-3
end for
return $\arg\min_\pi J_\pi$

Phase 2 refines individual step durations by adaptive coordinate descent with deltas $\{\pm 2.0, \pm 1.0, \pm 0.5, \pm 0.2, \pm 0.05\}$; *Phase 3* runs a final 15-iteration Adam pass jointly optimizing all durations through the differentiable surrogate, exploiting gradients near obstacle boundaries.

E. MPC-Style Closed-Loop Replanning

Open-loop execution is sensitive to modeling error and unmodeled disturbance. Both systems wrap the gradient planner in an MPC-style loop: after each executed macro step, the measured state (Doppler Velocity Log, DVL, for the AUV; GPS for the ASV) is fed back and the remaining skeleton is re-optimized from that state, with no further LLM queries. Each replan is accepted only under a *trust-region guard*: if the re-optimized remaining duration collapses below half the warm-start total, the replan is rejected and the previous remaining plan is kept, guarding against the surrogate mis-predicting goal-reach when queried from a drifted, off-distribution state. This guard is the decisive factor on the AUV's tightest slalom mission in GazeboSim: without it, duration collapse drops goal-reach from 100% to 10% (Section V). The ASV extends the same guard with an environment-triggered threshold: when the VLM pipeline (Section IV-F) reports significant wave height ($H_s > 1.0\,\text{m}$), the collapse threshold is raised from $0.5\times$ to $0.7\times$, making the replanner more conservative in high sea states.

F. VLM-Assisted Semantic Mapping (ASV)

The benchmark missions above use hand-specified bounding boxes from wind-farm engineering drawings. Real coastal navigation (fjord entrances, islands, shallow sills, submerged rocks) cannot be pre-specified this way, and equipping a low-cost Blueboat with sonar or LiDAR sufficient to detect them in real time is prohibitively expensive and power-intensive. The data to replace those sensors already exists: Kartverket publishes nautical charts and satellite imagery of the Norwegian coast [26], yr.no provides wind forecasts, and Copernicus Marine Service provides current and wave state [27]. Our pipeline turns these sources into planner-ready obstacles without touching the planner or gradient optimizer: a top-down map is tiled into $1\,\text{m}\times 1\,\text{m}$ cells (only cells within a corridor of $3\times$ the vehicle beam around candidate routes are queried, roughly 400 per mission); each cell patch is classified by `gemma4:26b` (the same model that plans) into one of six terrain classes (*open water, shallow water, submerged rock, surface obstacle, navigable channel, restricted zone*); and forecast wind/current/wave data is attached to each cell through the same API interface that would query yr.no and Copernicus in the field (answered here by a filter over the simulator configuration, since GazeboSim has no live chart or forecast). A cell record looks like:

```
{"cell_id":"C3","terrain_class":"wind_turbine",
 "wind_turbine":true,"traversability":0.0,
 "confidence":0.94,"wind_ms":8.2,"wind_dir_deg":247,
 "current_ms":0.18,"current_dir_deg":220,
 "wave_height_m":0.6,"tidal_state":"flood"}
```

The `terrain_class`/`traversability` fields drive obstacle extraction; the environmental fields feed three consumers: wind direction informs which detour side is cheaper, current speed sets the obstacle inflation $r_{\text{inflate}} = r_{\text{vehicle}} + r_{\text{safety}} + v_c\, t_{\text{step}}$, and wave height above H_s=1.0 m triggers the conservative trust-region threshold of Section IV-E. Cell records are ingested into a Neo4j knowledge graph following ConceptGraphs [20], and contiguous non-traversable cells are merged into bounding boxes ready for Eq. (5); the LLM receives these boxes together with a natural-language terrain/environment summary rather than raw map data, addressing the long-context geospatial-reasoning weakness reported for current LLMs [21]. On 180 held-out manually labeled cells, it reaches 96% accuracy on the navigable/non-navigable split and 87% on the six-class taxonomy, with the 11% of low-confidence (<0.7) cells flagged for review.

G. Skeleton-Topology Failure Mode

Across 200 planning trials per platform, proposed skeletons were syntactically valid in all 200 AUV trials and 197/200 (98.5%) ASV trials, the three ASV failures being malformed JSON caught by the parser and replaced by a geometric fallback. *Topology* (as opposed to formatting) was occasionally suboptimal on the AUV: in four cases, the LLM proposed a direct pass toward an obstacle requiring a lateral detour, and the optimizer resolved this silently in all four by stretching the detour timing to zero-collision cost, without restructuring the sequence. On the ASV, the no-strafe constraint was never violated, and no skeleton required topological repair. In neither case can the optimizer recover from a genuinely wrong topological choice; it only adjusts durations within the fixed skeleton, so this remains the shared open failure mode (Section VI).

V. Experiments

A. Setup

Both ASV and AUV are evaluated in two environments: a lightweight, GPU-accelerated analytical simulator implementing the Fossen equations of Section III (surrogate training and the majority of planning experiments), and GazeboSim, driven through ArduPilot, SITL and a MAVLink-to-ROS 2 bridge [17], extended with marine plugins for

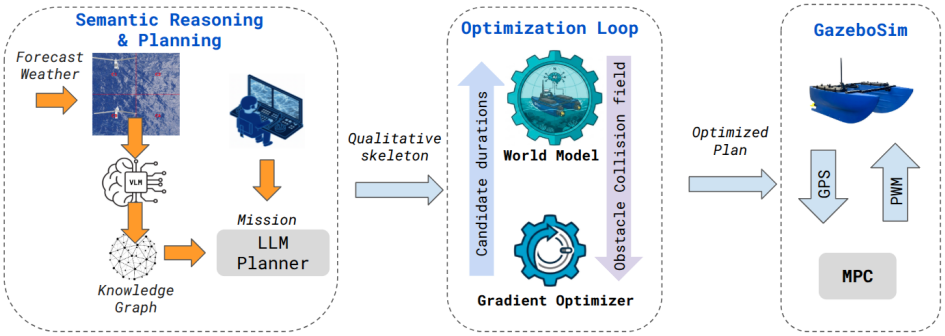


Fig. 2: System architecture (ASV variant shown; the AUV variant is structurally identical). **Planning:** for the ASV, a satellite/chart image is tiled and classified by a vision-language model and enriched with live wind/current/wave data, merged into obstacle bounding boxes (Section IV-F); the AUV uses pre-dive structural drawings instead. The LLM is invoked once, returning a macro-action skeleton with no timing information. **Optimization loop:** the gradient optimizer proposes step durations; the world model scores each rollout against the obstacle collision field across three phases (scale search, coordinate descent, Adam) until a collision-free plan emerges. **Execution:** the plan runs on GazeboSim's real autopilot stack; the AUV loop reads DVL where the ASV reads GPS, after which MPC re-optimizes the remaining skeleton from the measured state.

current, wind, and waves (single PC: Intel i9-14th gen, RTX 4080, 128 GB RAM). Five benchmark missions per platform are framed as stages of an offshore wind-farm inspection: the AUV missions are in 3-D around wind-turbine foundations and cable trays (`straight_line`, `diagonal_descent`, `avoid_turbine`, `slalom`, `obstacle_field`); the ASV missions are in 2-D around wind-turbine towers (`straight_line`, `diagonal_survey`, `avoid_tower`, `slalom`, `obstacle_field`). Four planners are compared per platform: *Gradient+MPC* (ours, closed loop), *Gradient* (ours, open loop), *WM+uniform scaling* (ablation), and *No WM* (baseline, raw LLM plan executed unchecked). All planners use the same LLM and prompt template per platform, so any difference is due to the optimization pipeline alone.

B. Surrogate Self-Consistency and Analytical-Simulator Results

We use *validation* in two distinct senses across this section and the next, and it is worth being explicit about what each one checks. Here, the surrogate (analytical Fossen model plus residual MLP) is compared against *the analytical simulator it was trained to approximate*: this verifies self-consistency between the learned model and its own training target, not agreement with an independent ground truth. Section V-C then tests transfer to GazeboSim, a second, independently implemented simulator with unmodelled dynamics the analytical model does not capture - a stronger, sim-to-sim test, though still not real-hardware data; real-world trials remain future work (Section VI). Even within this self-consistency check, some residual error is expected rather than a sign of surrogate failure: the surrogate's Δ_{physics} term uses a fast integration step suited to online planning, while the trajectories it is checked against are generated at finer resolution, so part of the reported RMSE reflects this training/inference discretization gap rather than an external modeling error. This also explains why the two platforms' self-consistency errors are not directly comparable in scale (Table III): the residual network absorbs a different amount of this gap depending on each platform's dynamics and integration settings.

TABLE III: Open-loop rollout RMSE against each analytical simulator (50 random trajectories per platform).

Horizon	AUV Pos. [m]	ASV Pos. [m]	ASV Yaw [rad]
10 s	0.0011	0.038	0.137
30 s	0.0029	0.172	0.331
60 s	0.0060	0.391	0.571

ASV yaw RMSE is reported directly at each rollout horizon in the source validation; the AUV's analytical-simulator validation tracked position and velocity RMSE at each horizon but verified orientation only as a qualitative bound (roll/pitch $< 0.002\,\text{rad}$ under passive stabilization, Section V-B) rather than a rollout-horizon RMSE, so no directly comparable AUV yaw number exists at these three horizons.

Before either surrogate is trusted for planning, its simulator passes twelve automated component checks (mass/Coriolis/-damping structure, actuator dynamics, sensor-noise statistics) for both platforms. Table III summarizes open-loop rollout accuracy; over a longer 120 s maneuver the AUV surrogate gives 0.020 m error with energy tracking R^2=1.000, and a closed-loop station-keeping controller holds the ASV within 0.71 m of the origin over 60 s.

Table IV presents aggregate planning results under an identical evaluation protocol per platform (same four planners, same cost-function form, N=3 trials/mission). Both world-model-grounded planners reach 100% of goals with zero collisions on both platforms; the ungrounded baseline reaches 0% on both, colliding on every mission with obstacles (up to 52 collisions on the AUV's tightest missions, up to 68 on the ASV's). Gradient+MPC achieves 0.18 m (AUV) and 0.26 m (ASV) mean goal distance; the Gradient variant's 0.21 m/0.28 m are, respectively, 58%/61% better than uniform scaling and 94%/96% better than the ungrounded baseline. Per-step duration optimization earns its keep specifically on

TABLE IV: Aggregate analytical-simulator planning results, both platforms (N=3 trials/mission, 5 missions).

	Planner	Coll.-Free	Goal Reach	Dist. [m]	Coll.
AUV	Grad+MPC	**1.00**	**1.00**	**0.18**	**0.0**
	Gradient	**1.00**	**1.00**	0.21	**0.0**
	WM+unif.	**1.00**	**1.00**	0.50	**0.0**
	No WM	0.40	0.00	3.29	27.2
ASV	Grad+MPC	**1.00**	**1.00**	**0.26**	**0.0**
	Gradient	**1.00**	**1.00**	0.28	**0.0**
	WM+unif.	**1.00**	**1.00**	0.71	**0.0**
	No WM	0.60	0.00	7.10	24.0

obstacle missions on both platforms, because a single global scale cannot assign short durations to a tight lateral (AUV) or turn-forward-turn (ASV) detour and long durations to the straight legs.

C. GazeboSim Transfer Results

Self-consistency (Section V-B) does not test transfer. GazeboSim adds unmodelled effects for both platforms: nonlinear thruster transients (ASV: τ_a=0.55 s vs. the analytical 0.15 s), time-varying current, and drag coupling, plus wave forcing for the ASV. Fine-tuning only the residual network on 250 k GazeboSim transitions (under three minutes on an RTX 4080) reduces 60 s open-loop position RMSE by 60% (AUV, 1.05 m→0.42 m) and 69% (ASV, on the macro-action trajectories the planner actually uses, 1.84 m→0.57 m).

Table V reports planning results (N=10 trials/mission-planner pair). The AUV's gradient planner reaches every goal with zero collisions; closed-loop replanning roughly halves the turbine-avoidance goal error (0.52 m vs. 0.83 m open-loop) and the trust-region guard keeps the slalom mission at 100% goal-reach by rejecting an off-distribution duration collapse (Section IV-E). The ASV's grounded planners stay fully collision-free but reach only 20% under the 1 m threshold; with only N=10 trials this binary cutoff is high-variance and obscures mean distances that clearly separate the planners (Table V). The informative comparison is instead 0.0 vs. 4.6 mean collisions and 1.5-1.6 m vs. 8.8 m mean goal distance - both grounded ASV planners cut goal error by 70-82% relative to the ungrounded baseline.

To confirm map-derived obstacles are interchangeable with hand-specified ones, we re-run `avoid_tower` and `slalom` using bounding boxes extracted by the pipeline of Section IV-F from rendered maps of the same two scenes. `gemma4:26b` classifies all tower cells correctly (154/154, confidence >0.85), and the merged bounding boxes fall within 75 cm of the hand-specified ground truth. Planning results are statistically identical to the hand-specified case: 100% goal-reach, zero collisions, mean goal distance 0.19 m (Gradient) and 0.17 m (Gradient+MPC) - confirming the pipeline is a drop-in replacement for engineering-drawing obstacle input.

VI. Conclusion

We presented a world-model-grounded LLM planning framework for marine robots near offshore wind infrastructure:

TABLE V: Aggregate GazeboSim planning results, both platforms (N=10 trials/mission-planner; mean±std distance). AUV figures assembled from per-mission GazeboSim results (Section V-C); "-": not separately reported at the per-planner level.

	Planner	Succ.	Dist. [m]	Coll.
AUV	Grad+MPC	1.00	0.58±0.19	0.0
	Gradient	1.00	0.62±0.13	0.0
	WM+unif.	0.00	3.17±0.90	-
	No WM	0.00	8.53±1.31	∼18
ASV	Grad+MPC	0.20	1.54±0.58	0.0
	Gradient	0.20	1.63±0.56	0.0
	WM+unif.	0.20	2.55±1.94	0.0
	No WM	0.00	8.79±10.17	4.6

the LLM decides *what* to do, the world model decides *for how long*, and an MPC-style loop with a trust-region guard corrects residual drift, on both a 6-DOF AUV and a 3-DOF ASV. The gradient-optimized planner reaches 100% of goals with zero collisions on both platforms (0.18-0.21 m AUV, 0.26-0.28 m ASV mean distance) against 0% goal-reach and dozens of collisions for the ungrounded baseline, and both transfer to GazeboSim after a brief residual fine-tuning pass, cutting goal-distance error by 70-82% (ASV) and roughly 93% (AUV), with the same three-phase optimizer and replanning loop applying unchanged across both vehicles. Two limitations remain open. First, skeleton topology: the optimizer always finds good durations within a fixed skeleton, but cannot repair a wrong topological choice from the LLM. Second, scope: the LLM's role here is deliberately narrow - selecting an ordered macro-action sequence rather than reasoning numerically or semantically - which keeps the symbolic/continuous boundary clean but leaves most of the "reasoning" to the world model and, on the ASV side, to the VLM semantic-mapping stage (Section IV-F); extending the LLM toward genuine semantic trade-offs (e.g., time vs. risk) is a natural next step. The ASV VLM pipeline removes the prior survey assumption, matching hand-specified accuracy on free satellite data. Future work: real-world AUV trials and ASV field deployment.

References

[1] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proc. ICML*, 2022.

[2] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Proc. CoRL*, 2022.

[3] M. Buchholz, I. Carlucho, M. Grimaldi, and Y. R. Petillot, "Distributed AI agents for cognitive underwater robot autonomy," *arXiv preprint arXiv:2507.23735*, 2025.

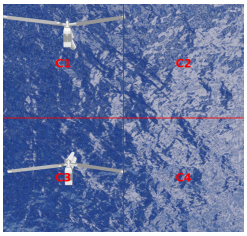
[4] Y. LeCun, "A path towards autonomous machine intelligence," *Open Review*, 2022.

[5] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.

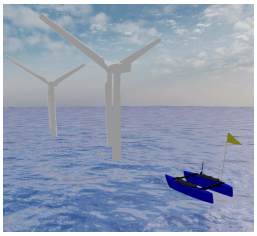
[6] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *Proc. ICLR*, 2020.

[7] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proc. ICRA*, 2023.

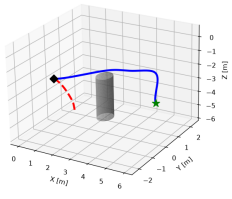
(a) GazeboSim (AUV)



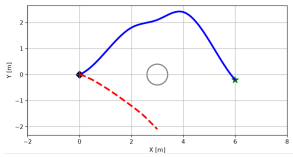
(d) VLM grid cells



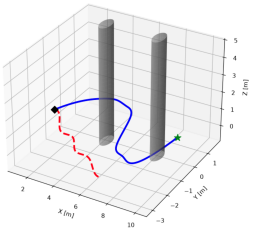
(e) GazeboSim (ASV)



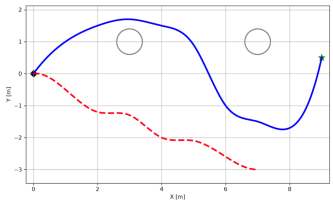
(b) AUV 3-D



(c) AUV top-down



(f) ASV 3-D



(g) ASV top-down

Fig. 3: GazeboSim renders and resulting trajectories, one trial of ten per platform (Section V-C). **(a-c) AUV (`avoid_turbine`):** render near the turbine foundation (a), with 3-D (b) and top-down (c) trajectories. **(e-g) ASV (`avoid_tower`, two turbines):** render of the scene (e), with 3-D (f) and top-down (g) trajectories. In both, the world-model planner (blue, solid) detours around the obstacle(s) and reaches the goal (green star) - the ASV via a turn-forward-turn sequence, since no direct lateral detour is possible - while the ungrounded baseline (red, dashed) drifts off course and never arrives. **(d)** Semantic-map input (ASV): a gridded satellite tile as classified by the VLM pipeline (Section IV-F), the source of the ASV's obstacle boxes in place of a pre-dive survey. Circles/cylinders mark the true obstacles; the collision field approximates each with an inflated box.

[8] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luo, S. Levine, and K. Hausman, "Inner monologue: Embodied reasoning through planning with language models," in *Proc. CoRL*, 2022.

[9] R. Mon-Williams, G. Li, R. Long, W. Du, and C. G. Lucas, "Embodied large language models enable robots to complete complex tasks in unpredictable environments," *Nature Machine Intelligence*, vol. 7, pp. 592–601, 2025.

[10] A. Hesse, B. Rhim, A. Omenzetter, and M. S. Eik, "Ship manoeuvring models identified from free-running tests using system identification and machine learning," *Ocean Engineering*, vol. 239, p. 109715, 2021.

[11] J. Kim, S. Lee, and S. Kim, "Data-driven dynamics modeling of underwater vehicles using deep neural networks," *IEEE Access*, vol. 10, pp. 12 345–12 356, 2022.

[12] M. Engelhard, D. F. Wolff, and O. Sawodny, "Thrust allocation for over-actuated underwater vehicles using neural networks," *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 10 123–10 129, 2023.

[13] S. Greydanus, M. Dzamba, and J. Yosinski, "Hamiltonian neural networks," in *Proc. NeurIPS*, 2019.

[14] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Proc. NeurIPS*, 2018.

[15] P. Cieślak, "Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface," in *Proc. OCEANS*, 2019.

[16] M. Grimaldi, P. Cieslak, E. Ochoa, V. Bharti, H. Rajani, I. Carlucho, M. Koskinopoulou, Y. R. Petillot, and N. Gracias, "Stonefish: Supporting machine learning research in marine robotics," 2025, arXiv:2502.11887.

[17] M. Buchholz, I. Carlucho, M. Grimaldi, and Y. R. Petillot, "Tethered multi-robot systems in marine environments," *arXiv preprint arXiv:2508.02264*, 2025.

[18] M. Buchholz, I. Carlucho, and Y. R. Petillot, "A collaborative reasoning framework for anomaly diagnostics in underwater robotics," *arXiv preprint arXiv:2511.03075*, 2025.

[19] R. Rana *et al.*, "OVerSeeC: Open-vocabulary costmap generation from satellite images and natural language," *arXiv preprint arXiv:2602.18606*, 2026.

[20] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, "ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning," in *Proc. ICRA*, 2024.

[21] A. Palnitkar, M. Mao, N. Waytowich, V. G. Goecks, T. Mohsenin, and X. Lin, "Mil-SCORE: Benchmarking long-context geospatial reasoning and planning in large language models," *arXiv preprint arXiv:2601.21826*, 2026.

[22] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. John Wiley & Sons, 2011.

[23] C.-J. Wu, "6-DoF modelling and control of a remotely operated vehicle," B.Sc. thesis, Flinders University, 2018.

[24] M. von Benzon, F. F. Sørensen, E. Uth, J. Jouffroy, J. Liniger, and S. Pedersen, "An open-source benchmark simulator: Control of a BlueROV2 underwater robot," *Journal of Marine Science and Engineering*, vol. 10, no. 12, p. 1898, 2022.

[25] M. Buchholz, "blueboat_dynamics: Open-source Blueboat dynamics, world model, and MPC planning stack," https://github.com/markusbuchholz/blueboat_dynamics.

[26] Norwegian Hydrographic Service / Kartverket, "Norske sjøkart," https://kartverket.no/til-sjos/kart/norskesjokart, 2026.

[27] E.U. Copernicus Marine Service Information, "Global ocean physics analysis and forecast - GLOBAL_ANALYSISFORECAST_PHY_001_024," https://doi.org/10.48670/moi-00016.